

AsymptoticAnalysis

After eighteen code reviews

Native request contracts, signed coordinate closure,
and Fourier recurrence termination

Independent technical audit

September 9, 2026 (Pacific time)

Reviewed repository: [VladimirReshetnikov/Asymptotic](#)

Pinned revision: 6687962f3c858a4f93623cfc496f33e35c6763d4

Abstract

This report evaluates the current AsymptoticAnalysis package against Wolfram Language's native expansion facilities, while separating additional findings from the eighteen reviews already archived in the repository. The principal additions are a silently ignored backend default, a held-option parser that rejects computed option keys, and an avoidable loss of ordered arithmetic for a signed reciprocal-square coordinate. A fourth contribution extends the previously identified homogeneous-recurrence termination principle to the Fourier engine, with a new native helper-level resource-failure witness and a tested candidate change.

The evidence combines inspection of the pinned sources and maintained review register, lexical screening of 45 review-source files, selected evaluations in Wolfram Language 15.0.0 on Linux, exact mathematical derivations, and 21 independent Python model/fixture checks. Native observations, mathematical proofs, synthetic patch tests and unexecuted regression specifications are identified separately. No new false analytic remainder bound is claimed. The report supplies focused source-patch candidates, a restricted chart pilot, reproducible observation and regression programs, and a development plan tied to these findings rather than a repetition of the existing backlog.

Evidence boundary. This is a revision-pinned incremental audit, not certification of every algorithm or a full-suite acceptance result. Two source changes were exercised separately on temporary standalone distributions. The complete combined patch, the full supplied native runner and the complete chart installer were not validated as integrated deliverables. No repository changes were made.

Contents

1	Executive assessment	4
2	Snapshot, method and nonduplication boundary	5
2.1	What was reviewed	5
2.2	How the eighteen earlier reviews were used	5
2.3	Evidence levels	6
3	Current architecture and mathematical contracts	6
3.1	A package germ is more than a truncated expression	6
3.2	The native result kind has a different meaning	6
3.3	Specialized engines and conservative fallback	7
3.4	Exact identities must precede asymptotic truncation	7
4	Comparison with Wolfram Language	7
4.1	Avoid an artificial “package versus built-ins” dichotomy	7
4.2	Normalize the task before comparing outputs	8
4.3	Native comparison results that are not claimed	9
5	N01: configured backend defaults are ignored	9
5.1	Public reproduction	9
5.2	Root cause	10
5.3	A narrow canonical-entry repair	10
5.4	The alias is a separate default-ownership problem	10
5.5	Impact and immediate workaround	11
6	N02: a computed option key is not prepared	11
6.1	The smallest contrast	11
6.2	Why the held parser misses it	11
6.3	Candidate preparation rule and its validation boundary	12
6.4	Required regression boundary	12
7	N03: a retained signed chart unnecessarily loses ordered closure	13
7.1	A public, mathematically elementary downstream failure of closure	13
7.2	The sign information is already present	13
7.3	The ordered result follows immediately	13
7.4	An independent quantitative bound for the same source	14
7.5	Restricted pilot and result	15
8	F01: Fourier recurrence termination still spends an irrelevant product	15
8.1	Attribution and scope	15
8.2	The coefficient recurrence	15

8.3	A three-row result that fails a budget of three	15
8.4	Repairing both termination conditions	16
8.5	Native candidate observation and independent cost model	16
9	A precise request model for the native boundary	17
9.1	Three facts that must not be conflated	17
9.2	Default resolution without accidental explicitness	17
9.3	Evaluation invariants	18
9.4	A diagnostic improvement tied to the findings	18
10	From the quadratic pilot to branch-aware coordinate closure	18
10.1	Solve a chart, not an unlabelled equation	18
10.2	A restricted family with a transparent proof	19
10.3	Regrading compatible small coordinates	19
10.4	Metadata and failure conditions	19
11	Validation strategy and actual execution record	20
11.1	What succeeded	20
11.2	Metamorphic tests specific to this audit	20
11.3	Patch and evidence hygiene	21
12	Performance and engineering recommendations tied to the findings	21
12.1	Measure avoided work, not just elapsed time	21
12.2	Do not turn the backend fix into unnecessary symbolic work	22
12.3	A chart fast path should eliminate, not hide, solver work	22
12.4	Build and regression scope	22
13	Prioritized development packages	22
14	Source-coverage map and findings not promoted	23
14.1	Coverage map	23
14.2	Examples of checks that did not become findings	24
15	Artifact guide and reproduction	24
16	Conclusion	25

1 Executive assessment

The package is most valuable as a structured calculus of real asymptotic germs, not as a replacement name for **Series**. Its distinctive contribution is the coupling of finite coefficients with a chosen small coordinate, a real branch, an explicit asymptotic error description, and selected refinement or verification data. Its newer native-backend mode solves a different problem: preserving native results outside the analytic representation without inventing an error theorem. These two roles coexist in the current public interface. The separation is mathematically appropriate, but it makes request interpretation consequential: selecting the wrong backend can change both the terms returned and the contract those terms carry.[1, 5, 6]

The strongest new public-interface finding is therefore not merely cosmetic. After a user sets the canonical function's default backend to **"Package"**, an omitted-selector call can still return a native result. Conversely, setting the default to **"Series"** leaves an elementary request on the package path, with its different cutoff convention. A second parser defect rejects a conventional computed option key even though a computed option container succeeds. Both are reproduced on the pinned distribution.

The main algebraic development opportunity is more specific than the already proposed enlargement of coefficient algebras. The package already has enough mathematics to multiply a Lerch expansion in $w = x^{-2}$ by x on the retained positive branch. Nevertheless, the generic coordinate inverter solves the chart equation without its domain and refuses the two unfiltered solutions. The operation then falls back to a valid but less useful composite result. This is an avoidable representational downgrade, not an incorrect bound.

The Fourier observation is an explicitly attributed delta to prior work. A helper computes the next power of its small perturbation before establishing that the next coefficient is zero. For a three-row exact answer, the unnecessary fourth candidate pair triggers the resource budget. The same structural principle was already recorded for the ordinary engine; the new contribution is the remaining Fourier location, a minimal native reproduction, the exact work analysis and a focused candidate repair.

ID	Classification	Observed consequence	Priority
N01	Public option-default defect	Backend defaults do not determine dispatch; the omitted-selector call can change representation and order semantics relative to the configured default.	High for API reliability
N02	Public held-parser defect	A computed key for "Backend" produces NativeOptionConflict ; literal keys and computed containers work.	Medium
N03	Closure and usability shortcoming	A branch-resolved reciprocal-square chart unnecessarily leaves ordered power-log arithmetic.	Medium
F01	Attributed performance/availability delta	Fourier unit composition spends a product budget after its coefficient recurrence terminates. A private native witness fails although its exact answer fits.	Medium; helper scope

The identifiers N01–N03 and F01 are local to this article; identifiers such as register P03 refer to the maintained upstream work map.

The immediate development recommendation is to make the backend selection obey its effective configuration, retain the conservative distinction between native and analytic objects, and implement narrow, proved chart fast paths before enlarging general transseries machinery. The Fourier change is a suitable independent repair, provided its zero-termination and support-exhaustion checks are tested together.

2 Snapshot, method and nonduplication boundary

2.1 What was reviewed

The audited revision is 6687962f3c858a4f93623cfc496f33e35c6763d4. The repository metadata place this checkpoint on September 10 in UTC, which is September 9 in the user's Pacific time zone. All source references in this article point to this immutable revision rather than moving `main`. The package at this checkpoint is named `AsymptoticAnalysis`; its public inverse constructor remains `AsymptoticInverse`, and its series-object head is `GeneralizedSeries`. Older report names and contexts must not be substituted mechanically into current examples.[1, 3]

The canonical implementation is modular under `src/Kernel`; the repository-root `AsymptoticAnalysis.wl` is a generated standalone distribution. Successful native observations in this audit loaded that standalone file through a download followed by local `Get`. Consequently, those observations test the current public distribution, not a hand-transcribed substitute for its kernels. Temporary candidate evaluations likewise loaded modified copies of that complete standalone distribution. No upstream files were edited.

The deepest inspection concerned the native request dispatcher, ordinary and composite arithmetic, the shared series representation, Fourier coefficients, the Dirichlet adapters and exact special-function normalization. The surrounding architecture was checked through source maps, focused source excerpts, the user-facing documentation and the maintained implementation register. A source-coverage map is provided in Section 14. This is not a claim that every line in all companion modules received an independent mathematical proof.

2.2 How the eighteen earlier reviews were used

The [wave-two index](#) and [implementation register](#) are essential context. The register consolidates 123 identified entries from eighteen review packages; their observations apply to earlier pinned snapshots, and many important repairs occurred afterward. A historical wrong result is not a current reproduction. Conversely, an earlier passing count does not validate a later package rename or a changed module.[3, 4]

For candidate-specific novelty screening, a successful Wolfram evaluation fetched 45 non-standalone `.tex` files beneath `external-reports/code-review`, including the split articles and reference files. It searched the literal strings `SetOptions`, `NativeCompatibility`, `seriesCoordinateRule`, `option key`, `option-key` and `Euler recurrence`. No matches were returned. This is a *lexical screen*, not proof that no author expressed a related idea differently. The semantic comparison rests on the register and relevant prior-topic summaries. A broader follow-up screen encountered a service failure and contributes no additional evidence.

Item	Nearest earlier material	Additional contribution here
N01	Current B01–B03 describe backend routing and contracts. Earlier review snapshots predate the present native dispatcher.	A configured selector is ignored, with native observations showing both missing native-order terms and loss of an intended package-only default. The alias issue is distinguished from the one-line canonical repair.
N02	B02 discusses held requests and computed containers.	A computed option <i>key</i> , unlike a computed container, is not prepared. The report isolates the parser mechanism and delayed-value obligations.
N03	X02 is a broad coefficient-closure proposal; C18 concerns incorrect recovery of target geometry.	The source object already retains the needed sign. The chart inverter simply does not use it. A concrete valid composite result can be strengthened to an ordered result, with an independent quantitative Lerch bound.
F01	P03 and the earlier ordinary Euler-recurrence termination finding.	The same principle still has a separate Fourier-engine application. The contribution is a new helper-level native failure, an exact pair-count witness and a separately exercised patch.

This report deliberately does not restate the established backlog as new discoveries. In particular, it does

not claim novelty for the previously recorded assumption leaks, semantic exponent grouping, parameter capture, native index limits, refinement precision planning, numerical cancellation, flat-sector grade loss, or general requests for complex sectors and multivariate transseries. The present roadmap is restricted to concrete consequences of N01–N03 and F01, with explicit links where it intersects existing work.

2.3 Evidence levels

There are four distinct evidence classes throughout the report. *Native observation* means that an identified expression produced a successful response from the connected Wolfram evaluator, using Wolfram Language 15.0.0 for Linux x86-64. *Mathematical proof* means a derivation independent of implementation output. *Model or fixture check* means an executed Python check of exact arithmetic, an independent recurrence model, or textual patch staging. *Supplied specification* means code or a test that is included for reproduction but whose complete execution is not claimed.

Network and service errors occurred during some additional probes. They are not package failures, and their intended outputs are not included as observations. The successful outputs are transcribed in `evidence/native_observations.json`; that file is not represented as an automatically exported full kernel log. The complete `run_observations.wl` and `desired_contracts.wlt` files were not run as units. Counts from upstream Windows validation records are historical context only and are not added to this audit's evidence totals.

3 Current architecture and mathematical contracts

3.1 A package germ is more than a truncated expression

A useful abstraction for the ordinary representation is

$$f(x) = b(x) + A(x) \left[\sum_{\beta < \rho} w(x)^\beta P_\beta(\log w(x)) + \mathcal{O}(w(x)^\rho (1 + |\log w(x)|)^k) \right], \quad w(x) \rightarrow 0^+. \quad (1)$$

Here the fixed parameters satisfy retained assumptions, the source follows a specified real approach, and b, A are exact retained factors where the representation permits them. In the implementation these roles appear as a variable, a positive scale variable, a logarithmic dummy variable, an offset, a prefactor, a coefficient jet and its precision. The public remainder can therefore contain the magnitude of the prefactor multiplying an inert `PowerLogRemainder`.[\[7, 9\]](#)

The finite approximation and its remainder are different pieces of information. Removing the remainder with `Normal` is useful for evaluation or display, but it is not an assertion that the finite expression equals the original function. Equally, a symbolic big- $\mathcal{O}$ descriptor need not be a computable bound valid at a given numeric point. The Dirichlet adapters are interesting precisely because they retain some explicit quantitative inequalities in addition to their asymptotic description.

The logarithmic degree matters. For $w \rightarrow 0^+$, $w^\rho \log^k w$ generally is not $\mathcal{O}(w^\rho)$ when $k > 0$. Conversely, for any fixed $\epsilon > 0$, it is $\mathcal{O}(w^{\rho-\epsilon})$. A sound library must distinguish a sharp logarithmic frontier from a deliberately weakened power-only bound. This is background for understanding the representation, not a new report of the earlier native import/export findings.

3.2 The native result kind has a different meaning

The current native dispatcher wraps the selected backend's result in `GeneralizedSeries` with "Kind" $\rightarrow$ "Native". It preserves the backend result, held request, recognized specifications, ambient assumptions and runtime provenance. Its analytic remainder and exactness fields are explicitly missing rather than guessed from whether the backend printed an $\mathcal{O}$ term. Analytic operations guard against treating such an object as an ordinary package jet.[\[6, 5\]](#)

This is the right direction. A native expression can contain a formal `SeriesData`, a condition, a list, an unevaluated native request or even an infinite representation. The absence of an explicit remainder in

that output does not justify attaching `Exact -> True`. Nor does accepting a complex native expression establish the hypotheses needed for a real Lipschitz error rule.

A shared wrapper head is nevertheless a usability hazard unless the kind is visible at the point of use. The same public constructor can return an analytic package object or a native-contract object, and an operation suitable for one can be rejected for the other. N01 shows why backend defaults must be treated as effective configuration rather than presentation preferences.

3.3 Specialized engines and conservative fallback

The source map separates coordinate construction and inverse-branch admission from ordinary coefficient calculus, retained Lambert/Gamma/Barnes cores, flat-sector and Fourier algebras, special-function ingress, refinement state, and numerical or interval checks. It is sensible that these engines do not all pretend to share one coefficient ring. A reciprocal-logarithmic core or a finite collection of exponential sectors carries structure that a simple polynomial-log jet does not encode.[1, 2]

When compatible approaches have no common ordered coefficient representation, composite arithmetic retains a finite expression and separate error envelopes. For $f = e_f + \mathcal{O}(R_f)$ and $g = e_g + \mathcal{O}(R_g)$, the implemented product uses

$$fg = e_f e_g + \mathcal{O}(|e_f|R_g + |e_g|R_f + R_f R_g). \quad (2)$$

This is a legitimate fallback. It prevents unsupported symbolic closure from turning into an unjustified single-scale precision claim. It also means that a conservative answer can still be unnecessarily weak *as a data structure*: N03 concerns a case where the same correct error could have remained in the ordered algebra.

3.4 Exact identities must precede asymptotic truncation

The special-function normalization module applies selected exact identities before finite expansions. For example, a half-integer modified Bessel expression may have a terminating dominant contribution while still containing the opposite exponential. The current source explicitly preserves the exact identity rather than inferring exactness from termination of one asymptotic part. Its hypergeometric and incomplete-Gamma identities likewise retain their stated parameter or domain restrictions. This is a positive architectural feature, not a newly detected defect.[12]

The broader design lesson is useful for F01: exact termination of a *coefficient recurrence* and apparent termination of a *dominant asymptotic sector* are different facts. The former can justify skipping further products when its recurrence is homogeneous. The latter does not, by itself, erase other sectors of the original function.

4 Comparison with Wolfram Language

4.1 Avoid an artificial “package versus built-ins” dichotomy

Native `Series` already supports Taylor, Laurent, selected fractional-power and logarithmic expansions, infinity, and successive specifications. Its output is usually `SeriesData`; unrecognized functions are treated as analytic by default unless the corresponding option changes that behavior. These are native capabilities, not features that originate in this repository.[13]

Native `Asymptotic` is broader than local polynomial expansion. Its documented inputs include functions, integrals, transforms and differential-equation expressions; its scale is inferred from the problem, and order `Infinity` can request a complete power-series representation. Its returned asymptotic order is not the package’s exclusive power cutoff.[15]

`InverseSeries` provides native series reversion. `AsymptoticSolve` is the more relevant comparator for implicit-function problems: it accepts specified solution centers, systems, real-domain selection and directional information. Calling the package an asymptotic inverse engine therefore does not establish that every problem it solves is beyond native Wolfram functionality.[16, 17]

The package’s strongest independent value is the organized, inspectable contract around admitted real generalized expansions, including specialized cores, logarithmic-degree bookkeeping, coefficient access, replay and selected certificates. Its native wrapper is interoperability, not a new derivation of the delegated coefficients.

Area	Native Wolfram position	Package position at the reviewed revision
Ordinary local series	Series and SeriesData supply mature expansion and manipulation facilities.	Explicit exclusive-cutoff semantics and retained analytic metadata; many elementary cases overlap.
Fractional, symbolic and logarithmic terms	Native representations can retain rational powers, logarithmic coefficients and external symbolic or rapid factors.[14]	Sparse real exponent/log-polynomial data, with an explicit logarithmic remainder degree in the admitted ordinary model.
Inverse expansions	InverseSeries reverts native series; AsymptoticSolve handles broader implicit problems.	Dedicated real branches, generalized source weights, specialized inverse cores and separately scoped residual/certificate facilities.
Gamma and related inverse cores	Native symbolic and asymptotic functions remain valuable building blocks.	Documented retained Lambert-core constructions for increasing Gamma-related inverses, plus Barnes and specialized adapters. This is endpoint- and model-specific.
Oscillatory and flat scales	Native asymptotic results may contain oscillatory or exponential factors.	Dedicated Fourier coefficients and finite flat-sector structures offer selected exact arithmetic; they are not unrestricted transseries support.
Zeta and Lerch forward tails	Native special-function facilities are available, but the contract must be checked per request.	Dedicated Dirichlet/geometric-moment derivations retain selected explicit forward tail bounds. N03 examines their downstream closure.
Realness and conditions	Native AsymptoticSolve supports a real domain; conditional results and assumptions depend on the selected engine.	Analytic package paths impose real admission and retain branch data. Native-kind output deliberately does not acquire these proofs.
Remainder operations	Native series arithmetic follows its representation’s formal rules.	Ordered operations propagate recorded errors; composite operations preserve conservative envelopes when ordered closure fails.
Numerical accuracy	Numerical solvers and arbitrary precision are not, by themselves, interval certificates.	The repository distinguishes diagnostics from quantitative inverse certificates; existing certificate issues and repairs remain tracked separately.
Input coverage	The union of Series and Asymptotic contains many native-specific forms.	Full native input-superset coverage is an accepted goal, not a completed theorem. Explicit delegation and selected automatic routes are implemented.
Evaluation/defaults	SetOptions changes defaults; OptionsPattern and OptionValue are the standard option mechanisms.[18, 19]	N01/N02 expose two deviations in the new held dispatch layer, rather than defects in the mathematics of the chosen backend.

4.2 Normalize the task before comparing outputs

A meaningful comparison specifies the source expression, parameters, expansion point, real or complex approach, branch, representation contract and requested precision. Merely passing the same integer to two functions is insufficient. In the audited package path, $\{x, 0, 3\}$ excludes x^3 ; native **Series** through

order three includes it. The following outputs were observed:

$$\begin{aligned}\text{Normal}(\text{package } \exp x) &= 1 + x + \frac{x^2}{2}, \\ \text{Normal}(\text{native Series } \exp x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6}.\end{aligned}$$

Neither expression is wrong for its own order convention. N01 is wrong because a user-configured backend does not select the expected convention. The documented collision of cutoff conventions is already acknowledged by the repository and is not presented here as a newly discovered bug.[5]

For performance comparisons, match the *achieved* approximation, not just the input order. A package request that returns an analytic remainder and retained branch data should not be declared slower merely because a native call returns a shorter unannotated expression. Conversely, calling native **Asymptotic** internally is not evidence of an independent package algorithm outperforming it. This audit does not claim a measured end-to-end speed advantage for either implementation.

4.3 Native comparison results that are not claimed

An attempted additional batch comparing polynomial and irrational inverse examples did not return a successful evaluator result. Accordingly, this report does not assert that the attempted native irrational inverse was unsupported, timed out, or disagreed with the package. The comparison above relies on official interface documentation, inspected package contracts and the successful elementary/native-routing observations. That is a narrower but reproducible basis than inferring capability gaps from a service error.

5 N01: configured backend defaults are ignored

5.1 Public reproduction

After loading the package, evaluate the following in a subsequent input. The source is deliberately elementary: the distinction cannot be attributed to a difficult special function or an undecidable branch.

```
SetOptions[AsymptoticExpansion, "Backend" -> "Series"];
s = AsymptoticExpansion[Exp[x], {x, 0, 3}];
t = AsymptoticExpansion[Exp[x], {x, 0, 3},
  "Backend" -> "Series"];
{Options[AsymptoticExpansion, "Backend"],
 s["Kind"], Normal[s], t["Kind"], Normal[t]}
```

The observed default list is {"Backend" -> "Series"}, but the two calls disagree:

Call	Kind	Finite expression
Selector omitted	Forward	$1 + x + x^2/2$
Explicit "Series"	Native	$1 + x + x^2/2 + x^3/6$

The omitted-selector result does not obey the configured selector. Its coefficients are correct for the engine that actually ran; the error is the effective request.

The reverse direction is more important for clients that require an analytic package contract:

```
SetOptions[AsymptoticExpansion, "Backend" -> "Package"];
a = AsymptoticExpansion[Exp[I x], {x, 0, 3}];
b = AsymptoticExpansion[Exp[I x], {x, 0, 3},
  "Backend" -> "Package"];
```

The first call returned a native object with finite expression $1 + ix - x^2/2 - ix^3/6$. The explicit package call was rejected with tag `InexactInput`. The point is not the diagnostic wording: the configured package-only default did not prevent native delegation. This is a semantic configuration failure, not a proof that the native expression itself is mathematically invalid.

5.2 Root cause

In `expansionDispatch`, the selector list is extracted from explicitly supplied held option trees. When that list is empty, the code chooses a literal `Automatic`:

```
backend = If[values === {}, Automatic,
            ReleaseHold[First[values]]];
```

This bypasses `Options[AsymptoticExpansion]` even though the public function advertises a backend option and allows its defaults to be changed. An empty list of explicitly supplied values means “use the current default”, not “use the default that happened to be present when this source was written”.[\[6, 18\]](#)

The bug has no complicated numerical preconditions. It applies whenever an omitted explicit selector is expected to inherit a non-`Automatic` backend default and the resulting routing differs. Native order and analytic-package admission make the difference visible, but the underlying configuration invariant fails even if the two finite expressions happen to coincide.

5.3 A narrow canonical-entry repair

The supplied candidate changes only the empty-selector branch:

```
backend = If[values === {},
            OptionValue[AsymptoticExpansion, {}, "Backend"],
            ReleaseHold[First[values]]];
```

This was exercised on a temporary modified standalone distribution. Setting the canonical default to `"Series"` then returned the native cubic expression. Setting it to `"Package"` then rejected the complex example just as the explicit package selector did. These are two selected observations, not a full regression suite.

The change intentionally leaves explicit-option precedence untouched: the first recognized explicit selector is still used. It also leaves the selected backend’s order convention untouched. Conflating this fix with a conversion between native order and package cutoff would enlarge the change and obscure its acceptance criterion.

The remaining tests should include invalid default selectors, delayed default values, explicit selectors overriding nonstandard defaults, duplicate selectors, nested option containers and calls through the alias. The supplied canonical fix should not be described as fully validated for those cases simply because the two basic counterexamples now pass.

5.4 The alias is a separate default-ownership problem

The held alias is implemented by forwarding its arguments to the canonical function, while its `Options` are copied from the canonical entry at initialization. A native observation showed that

```
SetOptions[AsymptoticExpand, "Backend" -> "Series"];
AsymptoticExpand[Exp[x], {x, 0, 3}]
```

still returned the package quadratic expression when the canonical function retained its `Automatic` default. The one-line canonical fix does not make the alias read its own defaults.[\[6\]](#)

Two coherent policies are available. Under a *single-owner alias* policy, the alias and canonical name share effective defaults and the API must not misleadingly advertise independent mutable configuration. Under an *independent-entry* policy, each name has its own effective defaults and both pass an owner identifier into the same held dispatcher. The latter can preserve `SetOptions` on either symbol, but it must be implemented for all exposed options rather than only the selector.

Blindly appending every default as an explicit rule is not a safe shortcut. Current automatic routing treats certain *explicit* package options as protective contracts. Materializing all defaults into explicit rules could force the package path on requests that previously delegated. Effective values and syntactic explicitness therefore need separate fields, as developed in Section 9.

5.5 Impact and immediate workaround

For callers that require predictable behavior before a repaired release, the operational workaround is to pass the backend explicitly at the call site or through an evaluated option container. An explicit "Package" selector is also preferable to assuming that a prior configuration change has disabled fallback. This is not an instruction to discard native interoperability; it is a way to select its contract deliberately.

N01 warrants a high API-reliability priority because it is silent and can change the object kind, accepted domain and order interpretation. It is not classified as a newly proved false asymptotic formula or an unsound interval certificate.

6 N02: a computed option key is not prepared

6.1 The smallest contrast

The following three forms should be distinguished carefully:

```
(* Literal key: succeeds. *)
AsymptoticExpansion[Exp[x], {x, 0, 3},
  "Backend" -> "Series"]

(* Computed container: succeeds. *)
opts = {"Backend" -> "Series"};
AsymptoticExpansion[Exp[x], {x, 0, 3}, opts]

(* Computed key: reproduced failure. *)
key = "Backend";
AsymptoticExpansion[Exp[x], {x, 0, 3},
  key -> "Series"]
```

The first two returned the native cubic expression. The third returned `NativeOptionConflict`. An isolated native control using an ordinary function with `OptionsPattern[]` and `OptionValue["Backend"]` accepted the computed key and returned "Series". The failing key form is therefore not intrinsically an invalid Wolfram option construction.

The result is an avoidable API incompatibility rather than a mathematical failure. It is particularly confusing for code generators: assigning a rule list to a symbol repairs the same request that fails when the rule is constructed directly inside the held call.

6.2 Why the held parser misses it

The dispatcher rightly avoids evaluating the entire source just to discover an option. It distinguishes literal requests from computed trailing containers. However, the preparation predicate declares every held rule to be already literal:

```
nativeComputedArgumentQ[
  HoldComplete[_Rule | _RuleDelayed]] := False;
```

At the same time, the selector extractor recognizes a literal string key "Backend". A symbol with an own value equal to that string is not recognized at this stage. The request can subsequently reach option-conflict logic with the wrong syntactic classification.^[6]

This is not a reason to replace the held parser with unrestricted recursive evaluation. Rules may occur inside the source or inside another option's value as ordinary data; those must not be reinterpreted as wrapper options. A delayed option value also must not be evaluated simply to determine its key.

6.3 Candidate preparation rule and its validation boundary

The supplied candidate distinguishes literal string keys, symbols with own values and other computed key expressions:

```
nativeComputedArgumentQ[
  HoldComplete[(Rule | RuleDelayed)[key_Symbol, _]] :=
  OwnValues[key] != {};

nativeComputedArgumentQ[
  HoldComplete[(Rule | RuleDelayed)[_String, _]] := False;

nativeComputedArgumentQ[
  HoldComplete[_Rule | _RuleDelayed]] := True;
```

This reuses the existing preparation route instead of adding an unrelated evaluator. A literal symbol such as `Assumptions`, without an own value, remains literal. A key expression can be prepared once before selector classification. An ordinary `RuleDelayed` retains its delayed right-hand side during that preparation.

The mechanism received an isolated native check. A stored key was classified as computed, a literal string key was not, and an expression of the form

```
(n++; "Backend") :> (m++; "Series")
```

advanced the key counter once while preparation left the value counter at zero. Releasing the extracted held value advanced the latter counter once. These observations support the mechanism, but **the complete package with this patch did not receive a successful integrated evaluation**: the attempted calls failed at the connector/service level. The candidate is therefore not reported as a validated package repair.

6.4 Required regression boundary

The important invariant is selective, single consumption. A pure computed key and its evaluated literal form should have the same effective meaning. A delayed backend value should be consumed once. A rule buried in the source, or in an option value, must remain data. A specification rule such as `x -> 0` must not be transformed into an option merely because both constructs use `Rule`. An unresolved or invalid key must produce a controlled diagnostic without repeatedly executing a program that failed to resolve it.

The existing computed-container success is a useful regression control. It should not be broken by a key repair. The supplied desired-contract tests include the ordinary computed-key case and a counter-based case; the broader language of valid option trees still requires native acceptance before this candidate is integrated.

7 N03: a retained signed chart unnecessarily loses ordered closure

7.1 A public, mathematically elementary downstream failure of closure

Consider the forward expansion

```
s = AsymptoticExpansion[LerchPhi[1/2, 2, x^2],
  {x, Infinity, 5}, "Backend" -> "Package"];
p = SeriesMultiply[s, x];
```

The native observation produced

$$\text{Normal}(s) = \frac{2}{x^4} - \frac{4}{x^6} + \frac{18}{x^8}, \quad R_s = \text{PowerLogRemainder}(x^{-2}, 5, 0). \quad (3)$$

The adapter is implemented through geometric moments in the retained reciprocal argument.[11] The input object's scale was **PowerLog**. The product's scale was **Composite**, with the finite expression multiplied by x and error

$$|x| \text{PowerLogRemainder}(x^{-2}, 5, 0). \quad (4)$$

That error is sound. There is no counterexample to the composite envelope formula here. The shortcoming is that the object has lost an ordered precision structure that is already available from the retained branch.

For comparison, the otherwise analogous Lerch expansion with argument x remained **PowerLog** after multiplication by x . Its finite approximation was $2/x - 4/x^2 + 18/x^3$, with a reciprocal-coordinate remainder of power four. Thus this is not a general lack of regular-operand multiplication for the adapter.

7.2 The sign information is already present

The nonlinear source object recorded a target approach to $+\infty$. More importantly, its internal series domain in the successful inspection included $x^{-1} > 0$ together with $x^2 > 0$. Native simplification confirmed that this domain implies $x > 0$. The repair does not require assuming an unrecorded sign to make the answer convenient.

Nevertheless, **seriesCoordinateRule** calls a real **Solve** on the equation $w(x) = u$ without including the retained domain, and it accepts only a unique returned rule. For $w = x^{-2}$,

$$x^{-2} = u, \quad u > 0 \quad \implies \quad x = \frac{1}{\sqrt{u}} \quad \text{or} \quad x = -\frac{1}{\sqrt{u}}. \quad (5)$$

Without the domain the solutions are genuinely ambiguous. On the retained positive branch they are not. A direct helper call with the reciprocal-square coordinate and explicit domain $x > 0$ still returned **\$Failed**, confirming that the helper did not exploit that information.[7]

The regular-operand path first tries to express the exact operand in the recorded coordinate. When this conversion fails, the surrounding arithmetic can fall back to the composite engine. The fallback is conservative, but the failure of the exact conversion is avoidable.[8, 9]

7.3 The ordered result follows immediately

On the positive branch, $x = w^{-1/2}$. Multiplying a row $w^\beta P(\log w)$ by x shifts its exponent to $\beta - 1/2$, without enlarging its logarithmic degree. Therefore the product in (3) can be represented as

$$x \Phi(1/2, 2, x^2) = 2w^{3/2} - 4w^{5/2} + 18w^{7/2} + \mathcal{O}(w^{9/2}), \quad w = x^{-2}, \quad x \rightarrow +\infty. \quad (6)$$

The notation Φ denotes Lerch's transcendent. This is an ordinary rational-power jet in exactly the package's existing coordinate. No new reciprocal-log algebra, Fourier mode, multi-rate transseries or analytic function theorem is needed.

Lemma 7.1 (Exact monomial multiplication). *Let $w \rightarrow 0^+$ and*

$$f = \sum_j c_j w^{\beta_j} + \mathcal{O}(w^\rho(1 + |\log w|)^k).$$

For a fixed real α and a fixed nonzero real c ,

$$cw^\alpha f = \sum_j cc_j w^{\beta_j + \alpha} + \mathcal{O}(w^{\rho + \alpha}(1 + |\log w|)^k).$$

The hidden constant changes by at most the fixed factor $|c|$.

Proof. Multiply the defining remainder inequality by $|c|w^\alpha$, which is nonnegative for positive w . The finite part transforms exactly. No differentiation or interchange of asymptotic limits is involved. $\square$

The lemma applies with $\alpha = -1/2$, $c = 1$. On the negative source branch, $c = -1$ changes the finite coefficients' signs, but the error magnitude has the same exponent shift. The sign must be proved from the branch data; choosing the positive radical unconditionally would turn a closure improvement into a correctness defect.

7.4 An independent quantitative bound for the same source

The above exponent shift already proves the required asymptotic transport from the stored contract. It is also possible to verify the source and its bound without trusting the package's coefficient generation. For $a > 0$, the defining convergent Lerch sum gives

$$\Phi(1/2, 2, a) = \sum_{n=0}^{\infty} \frac{2^{-n}}{(a+n)^2} = a^{-2} \sum_{n=0}^{\infty} 2^{-n} \left(1 + \frac{n}{a}\right)^{-2}. \quad (7)$$

The defining-function reference is DLMF Section 25.14.[20]

The following exact identity is useful:

$$\frac{1}{(1+t)^2} - (1 - 2t + 3t^2) = -\frac{t^3(4+3t)}{(1+t)^2}, \quad t \geq 0. \quad (8)$$

Since $(4+3t)/(1+t)^2 \leq 4$, the absolute remainder is bounded by $4t^3$. The required geometric moments are

$$\sum_{n \geq 0} 2^{-n} = 2, \quad \sum_{n \geq 0} n 2^{-n} = 2, \quad \sum_{n \geq 0} n^2 2^{-n} = 6, \quad \sum_{n \geq 0} n^3 2^{-n} = 26. \quad (9)$$

These identities follow by repeated application of $z d/dz$ to $(1-z)^{-1}$ and evaluation at $z = 1/2$. Absolute convergence justifies that computation inside $|z| < 1$.

Proposition 7.2 (A bound for the audited Lerch product). *For every $x > 0$,*

$$\left| x\Phi(1/2, 2, x^2) - \left(\frac{2}{x^3} - \frac{4}{x^5} + \frac{18}{x^7} \right) \right| \leq \frac{104}{x^9}. \quad (10)$$

The difference inside the absolute value is nonpositive.

Proof. Insert (8) with $t = n/a$ into (7). The first three moments give $2a^{-2} - 4a^{-3} + 18a^{-4}$. The remainder is nonpositive term by term, and its absolute value is at most $4a^{-5} \sum_{n \geq 0} n^3 2^{-n} = 104a^{-5}$. Put $a = x^2$ and multiply by positive x . $\square$

The explicit constant is not a claim that the package currently transports its quantitative-bound metadata through this operation; that is a separate, already recorded development topic. Here it independently establishes that the proposed ordered result is describing the original special function, not merely the finite polynomial in its metadata.

7.5 Restricted pilot and result

A focused native pilot added a more specific private chart rule: when the coordinate is exactly x^{-2} and the retained domain proves $x > 0$, return $x \mapsto u^{-1/2}$. In the exercised calculation the operand's domain was also conjoined with the already implied condition $x > 0$; native simplification had established that this adds no new restriction to that case.

The product then remained `PowerLog`; its remainder was `PowerLogRemainder[x^(-2), 9/2, 0]`. Its finite expression was written using powers of x^{-2} , and `FullSimplify` under $x > 0$ returned zero for its difference from $x \text{ Normal}(s)$. This validates the positive-branch mechanism for the witness.

The shipped `QuadraticChartPilot.wl` packages that idea with a negative-branch rule and a saved-definition restoration function. The complete installer, negative-branch case and restoration lifecycle were not exercised together during the audit. It is a restricted candidate for disposable kernels, not a general coordinate solver or a supported upstream extension. Section 10 describes the integration obligations.

8 F01: Fourier recurrence termination still spends an irrelevant product

8.1 Attribution and scope

The ordinary-engine homogeneous recurrence was already the subject of earlier reviews and register item P03. The current register records an ordinary-engine repair. The Fourier helper `fourierComposeBlock` has a separate implementation with the analogous ordering problem still present. This section claims an additional location and concrete evidence, not discovery of the general early-termination principle.[3, 10]

The demonstrated failure is a *private-helper native failure*. Public Fourier construction and residual checks provide the surrounding use context, but this audit has not demonstrated a small public constructor call that reaches exactly the same failure with its own user-selected budget. That distinction matters when prioritizing the issue.

8.2 The coefficient recurrence

Let U be a positive-valuation perturbation, let ℓ denote the logarithmic variable, and let $C(\ell)$ be a finite Fourier-polynomial coefficient. The expansion of

$$(1 + U)^p C(\ell + \log(1 + U)) = \sum_{j \geq 0} c_j(\ell) U^j \quad (11)$$

uses

$$c_0 = C, \quad c_{j+1} = \frac{(D_\ell + p - j)c_j}{j + 1}. \quad (12)$$

For a mode $P_\omega(\ell)e^{i\omega\ell}$, the derivative is $(P'_\omega + i\omega P_\omega)e^{i\omega\ell}$, exactly the operation implemented by the Fourier Euler helper.[10]

Lemma 8.1 (An absorbing zero). *If a coefficient c_m in (12) is identically zero under the retained assumptions, every c_j for $j \geq m$ is identically zero. Therefore no power U^j with $j \geq m$ is needed to compute (11).*

Proof. Each subsequent step applies a linear differential operator to zero and divides by a nonzero positive integer. Induction gives the conclusion. This implication concerns the homogeneous recurrence; it does not apply to an arbitrary externally supplied Taylor-coefficient generator, where isolated zero coefficients may be followed by nonzero ones. $\square$

8.3 A three-row result that fails a budget of three

Take only the zero Fourier frequency, with constant amplitude one. Let $p = 1$ and

$$U = w + w^2, \quad h = 5.$$

Then (11) is exactly $1 + w + w^2$. The coefficients are $c_0 = 1$, $c_1 = 1$ and $c_2 = 0$. The following native helper call failed:

```
AsymptoticAnalysis'Private'catch[
  AsymptoticAnalysis'Private'fourierComposeBlock[
    {{1, {{0, 1}}}, {2, {{0, 1}}}},
    1, {{0, 1}}, 5, ell, True, 3, 1]]
```

The result was `Failure["ResourceLimit", ...]` with message indicating that Fourier jet multiplication exceeded the retained-pair budget.

The input support has two rows; the exact result has three. The first product $1 \cdot U$ has two retained pairs. The next product U^2 has four retained pairs below the exclusive cutoff five:

$$(1, 1), \quad (1, 2), \quad (2, 1), \quad (2, 2).$$

That product exceeds the budget of three even though its coefficient is zero. The current loop forms the product before computing that next coefficient and discovering termination. The failure is thus not evidence that the requested exact answer intrinsically exceeds the stated support or useful-product budget.

8.4 Repairing both termination conditions

There are two distinct reasons to stop: the next coefficient is identically zero, or the next product cannot have any retained support. Reordering only the coefficient calculation can create unnecessary coefficient work in the second case. The candidate therefore checks the next minimum weight first, then computes the next coefficient, and only then spends the product budget.

In outline, after the already existing input checks:

```
If[! less[product[[1, 1]] + u[[1, 1]], cutoff],
  Break[]];
coefficient = nextHomogeneousCoefficient[coefficient, k];
If[coefficient == {}, Break[]];
If[k >= limit, fail[...]];
product = fourierJetMul[product, u, cutoff, ...];
If[product == {}, Break[]];
(* Accumulate coefficient times product, then increment k. *)
```

The executable patch uses the original Fourier coefficient operations rather than the schematic helper name above. The minimum-weight check is justified by sorted positive-valuation support: every next product weight is at least the sum of the two minima. The candidate relies on the existing comparison and canonicalization contracts; it does not repair previously identified unrelated comparator or resonance issues.

This changes which useless continuation is avoided, and can alter resource-failure ordering at termination. That is intentional, but it must be tested against valid nonterminating Fourier examples, frequency budgets, exact support exhaustion and assumption-dependent coefficient zeros. A performance repair should not weaken the admission checks on malformed inputs or silently discard modes.

8.5 Native candidate observation and independent cost model

The candidate was applied to a temporary full standalone distribution. The same helper call returned exactly

```
{{0, {{0, 1}}}, {1, {{0, 1}}}, {2, {{0, 1}}}}
```

A public control, `AsymptoticFourierInverse[x+x^2,...]` with cutoff seven and `"MaxTerms" -> 7`, still produced a residual report with `ZeroBelowCutoff -> True`. That report concerns exact formal

composition of its retained finite Fourier forward model. It is not a certificate for an unspecified input remainder, and the patch was not tested on the full Fourier suite.

For a perturbation with m distinct weights and a cutoff retaining every pairwise sum, the unnecessary square generates m^2 candidate pairs. The useful constant-plus-perturbation answer has only $m + 1$ rows. The avoidable work is therefore quadratic in the number of perturbation rows in this family, even though the requested function is linear in that perturbation.

The independent Python recurrence model reproduces the two-row failure and its elimination. An eight-row deterministic case records exactly 64 avoided pairs. These are exact combinatorial counts, not measured Wolfram wall-clock speedups. Coefficient simplification, allocation and frequency convolution can have additional costs that the model intentionally does not estimate.

9 A precise request model for the native boundary

9.1 Three facts that must not be conflated

N01 and N02 expose different layers of one boundary. The dispatcher needs to know which public entry owns the defaults, what effective value an option has, and whether that value was explicitly supplied. These are not equivalent facts. A fourth fact—whether a delayed value has already been consumed—is an evaluation property rather than an option value.

A concrete internal request record could contain the following fields. These are proposed implementation fields, not claims about an existing public API.

Proposed field	Required meaning
<code>EntrySymbol</code>	The public entry whose default policy applies. This resolves the alias ownership question rather than guessing from the final canonical call.
<code>OriginalHeldArguments</code>	Original syntax for diagnostics and provenance; it is not the mutable effective configuration.
<code>PreparedSource</code>	The source form actually passed onward, retained so a fallback does not rerun an already consumed source program.
<code>ExplicitOptionKeys</code>	Keys supplied by the caller, after permitted key preparation. Used by explicit-contract protection without pretending every default was explicit.
<code>EffectiveBackend</code>	The selected value after explicit-option precedence or default resolution. N01 requires this to come from the effective configuration.
<code>ResolvedCommonOptions</code>	Common values that a package attempt has already consumed and that may be reused by an allowed native fallback.
<code>SelectedOrderConvention</code>	The convention actually applied, rather than an unqualified integer called order or cutoff.
<code>ResultContract</code>	Native formal/asymptotic or package analytic; this must be determined by the executed route, not by the constructor's shared head.

This proposal does not supersede the repository's existing native-contract plan. Its additional purpose is to make the default-ownership and computed-key defects unrepresentable in the dispatcher: the effective selector cannot silently revert to a literal `Automatic`, and a key is not classified before the preparation phase that its syntax requires.

9.2 Default resolution without accidental explicitness

A tempting fix is to join all `Options[entry]` to the supplied argument list. That makes `OptionValue` convenient, but also changes which options are syntactically present. Under the current routing policy, an

explicit direction or package resource contract prevents inappropriate fallback. A default with the same value need not have the same syntactic role in native compatibility.[5, 6]

A safer procedure is to classify explicit keys first, resolve the backend from the owning entry only if no explicit selector is present, and keep default-resolved values separate from explicitness flags. The policy for non-backend defaults should then be specified independently: some belong to the wrapper, some to the package engine, and some deliberately remain native backend defaults. Exposing an option in the wrapper’s `Options` should not leave its ownership ambiguous.

For an alias with independent options, passing an entry identifier into the held dispatcher avoids evaluating and injecting every default early. For a true single-owner alias, documentation and configuration access should consistently identify the canonical owner. Supporting both interpretations accidentally is worse than choosing either one explicitly.

9.3 Evaluation invariants

The following invariants are narrowly motivated by the reproduced defects and by the dispatcher’s stated contract:

1. For a pure request with no explicit backend, changing the owning entry’s backend default changes the effective selector exactly as an explicit selector would, apart from original-syntax provenance.
2. A computed option key is prepared once; a delayed value is not consumed merely to identify that key.
3. A fallback reuses any source or common option already consumed by the package attempt. Original programs are not silently replayed to reconstruct metadata.
4. The branch, kind and actual order convention of the result agree with the executed route. A native-kind object is not promoted to analytic exactness because it lacks a displayed remainder.

The first two are the new acceptance targets here. The latter two preserve existing design work while repairing the boundary. They are not new allegations that all current fallback calls reevaluate their source or invent exactness.

9.4 A diagnostic improvement tied to the findings

An optional held request-explanation facility would be useful only if it reports a genuinely prepared request without forcing the source or delayed options a second time. Its output should explain, for example, “backend inherited from canonical default”, “explicit selector”, “native-only option route” or “permitted representation fallback”. The distinction between a requested default and an actual route would make N01 immediately visible in tests and notebooks.

This is a concrete observability proposal, not a reason to duplicate the package’s existing metadata wholesale. It should reuse the request record already needed for execution. A preview that executes the request independently would introduce precisely the evaluation ambiguity that the parser is trying to avoid.

10 From the quadratic pilot to branch-aware coordinate closure

10.1 Solve a chart, not an unlabelled equation

The problem in N03 is not that a real solver returns two solutions. Both solutions in (5) are correct for the equation it was given. The missing information is the selected branch of the *chart*. A coordinate is a map plus an approach and a domain, not just an expression that happens to tend to zero.

A chart record sufficient for the witness would retain $w = x^{-2}$, $x \rightarrow +\infty$, $w \rightarrow 0^+$, the proved local domain $x > 0$, and inverse rule $x = w^{-1/2}$. On the opposite branch the inverse rule is $x = -w^{-1/2}$. Their finite expressions differ, while the magnitude of the coordinate Jacobian or a multiplicative error factor may be the same. Collapsing the two rules before branch selection is therefore not harmless.

The existing domain already proves the sign in the witness. A first integration step can consequently be much smaller than a general atlas: add bounded, domain-checked rules for reciprocal squares, with both

signs and a conservative refusal when neither is proved. The pilot demonstrates this mechanism. It should not be generalized by selecting `First[Solve[...]]`.

10.2 A restricted family with a transparent proof

A useful next family is

$$w = c[\sigma(x - a)]^q, \quad c > 0, \quad \sigma \in \{-1, 1\}, \quad \sigma(x - a) > 0, \quad q \neq 0, \quad (13)$$

with fixed admitted real parameters. Its inverse is

$$x = a + \sigma(w/c)^{1/q}, \quad w > 0. \quad (14)$$

The positive base makes the real power unambiguous. The sign, endpoint and direction must be proved to match the original source approach; the sign of q determines whether this chart shrinks near a finite origin or at large distance. A rule that only verifies the algebraic composition identity can still select the wrong approach.

This family covers more than the exact reciprocal-square spelling without invoking a general symbolic solver. Parameter signs must remain explicit; uncertain c or an unproved branch should preserve the existing conservative fallback. For charts beyond this family, a bounded solver may supply candidates, but acceptance must verify the coordinate identity, eventual domain, approach and uniqueness of the selected local branch. Conditions or generated parameters cannot simply be discarded.

10.3 Regrading compatible small coordinates

Some apparent changes of coefficient algebra are merely changes of small coordinate. Suppose both w and v tend to zero positively and

$$w = cv^q, \quad c > 0, \quad q > 0 \quad (15)$$

with fixed data. Then

$$w^\beta P(\log w) = c^\beta v^{q\beta} P(\log c + q \log v). \quad (16)$$

The coefficient remains a polynomial in $\log v$. Also

$$\mathcal{O}(w^\rho(1 + |\log w|)^k) = \mathcal{O}(v^{q\rho}(1 + |\log v|)^k), \quad (17)$$

where the two sides denote equivalent error classes up to fixed positive constants. To prove this, use $w^\rho = c^\rho v^{q\rho}$ and the two-sided comparability of $1 + |\log c + q \log v|$ with $1 + |\log v|$ for fixed c, q .

The positive q requirement in (15) is essential: it is a relation between two *small* coordinates. It is not the same sign restriction as the source-chart exponent in (13). Confusing these roles can reverse a cutoff or an approach.

This gives a bounded extension to the current ordered algebra: branch-aware monomial regrading before falling back to composite envelopes. It is a sharper, implementable subcase of the broad coefficient-closure discussion already tracked as X02, not a claim to have implemented arbitrary common transseries scales.

10.4 Metadata and failure conditions

A successful chart conversion should preserve the original source and branch, state the new coordinate, transform the cutoff and remainder consistently, and keep enough information to interpret later coefficient requests. A derivative contract should be transported only under its stated coordinate convention; otherwise retaining a conservative lower derivative order is safer than assuming every chart transformation preserves it automatically.

Refinement creates an additional integration obligation. A locally modified object that has an unrecognized recipe may have correct present coefficients but no replayable source path. The pilot does not implement a general chart-refinement protocol. Production integration should either retain an original constructor plus a replayable chart transform or explicitly decline refinement of an unsupported derived chart. The existing general refinement precision issue is already recorded and is not reintroduced here as a new finding.

Performance also matters. Repeatedly solving the same sign problem at every scalar multiplication is unnecessary once a chart’s local sign has been proved and retained. The bounded chart construction can carry that evidence forward under compatible assumptions. This is a specific use of proof retention for N03, not a new blanket recommendation to add global caches.

11 Validation strategy and actual execution record

11.1 What succeeded

Selected public baseline observations established N01, N02 and the composite fallback in N03. The private Fourier witness established F01 at helper scope. A temporary canonical-default patch repaired its two selected default cases. A temporary Fourier patch repaired its helper witness and preserved one public residual control. A restricted positive reciprocal-square rule enabled the ordered Lerch product, whose finite expression was verified against the original finite approximation under the branch assumptions.

The isolated computed-key parser experiment also succeeded, including the key/value evaluation counters. It does not substitute for a full native run of the modified package. The extended chart installer, its negative-branch path, restoration and the complete combined set of candidates were not exercised as integrated artifacts.

The local independent checks ran successfully: **21 test methods, zero failures and zero errors**. They check exact rational identities and bounds, monomial error transport, a zero-frequency recurrence/cost model, comparison with an independent polynomial-power oracle, and safe textual patch fixtures. They do not load or emulate the Wolfram package. Their short runtime is not a package benchmark.

Artifact or observation	Evidence obtained	Not established
Pinned public distribution	Selected routing, computed-key, Lerch and control outputs in a real Wolfram kernel.	Full test-suite acceptance or all-platform behavior.
Canonical backend patch	Configured Series produces the native cubic; configured Package rejects the complex witness.	Alias defaults, every delayed/default case or combined-patch acceptance.
Computed-key candidate	Isolated native rule classification and single key/value consumption.	Successful full-package execution of this candidate.
Fourier candidate	Exact three-row helper output and one zero-residual public control.	All Fourier budgets, every frequency pattern, or an end-to-end speedup.
Quadratic chart mechanism	Positive-branch ordered product and exact equality of finite expressions under its domain.	Full installer lifecycle, negative native branch or general chart completeness.
Python checks	21 exact-model/fixture methods passed.	Wolfram semantics or package regression acceptance.
Supplied native runner and tests	Reproducible observation program and desired-contract specifications.	A claimed aggregate passing result for either complete file.

11.2 Metamorphic tests specific to this audit

A useful expansion test compares meaning across equivalent requests, not only isolated expected strings. For N01, compare a pure omitted-selector call under a changed default with an explicit-selector call, then restore the default reliably. Compare object kind, executed backend and normalized mathematical content; held provenance is expected to differ.

For N02, compare literal keys, stored keys, evaluated containers and generated keys, while retaining counters on deliberately delayed values. A metadata comparison must not trigger the source or delayed

values again. Include rules inside source expressions as negative parser controls: they are not options even when their keys happen to equal "Backend".

For N03, compare a direct ordered construction with a chart-resolved product and verify finite-expression equality under the retained branch. Compare the resulting *error classes* rather than only their printed syntax. A power of x^{-2} on $x > 0$ can represent the same function as a power of x^{-1} while displaying differently. Include a negative branch, an unproved sign, a contradictory domain and a parameterized positive scale.

For F01, test two orthogonal stopping conditions: exact recurrence termination and exhausted support at the requested cutoff. Include a nonterminating Fourier amplitude to ensure the coefficient check does not introduce an unjustified stop. Include assumption-dependent zero coefficients only with their assumptions retained, and distinguish a zero coefficient from one that a simplifier failed to decide.

These tests are additional, narrow acceptance obligations. They are not a replacement for the existing suites guarding assumption retention, native admission, logarithmic frontier precision, composition scope or certificates.

11.3 Patch and evidence hygiene

The patch stager verifies that each requested baseline anchor occurs exactly once before writing a new output. It refuses in-place modification, preexisting output paths, duplicate patch names, missing anchors and duplicate anchors. An anchor match is a safety check for the transformation, not authentication of an entire source tree; the user should supply the pinned source specified in this report.

The combined candidate is not automatically certified by staging it. Native changes were exercised separately, while the computed-key candidate has only mechanism-level native evidence. The output of `stage_patches.py` says so explicitly. The supplied tests intentionally express desired contracts that some baseline cases violate; a baseline failure in that suite is not evidence that the test file itself was supposed to be an acceptance record.

The archive has no checksum files. Revision identifiers and source URLs identify the reviewed material. The archive also avoids redistributing the large upstream standalone or all historical reports; the observation program can acquire the pinned standalone directly, while the corpus script can reproduce the lexical screen in a network-enabled environment.

12 Performance and engineering recommendations tied to the findings

12.1 Measure avoided work, not just elapsed time

F01 provides a useful example of a performance assertion that can be established without a noisy timing experiment. For $p = 1$ and m retained perturbation rows, the result has $m + 1$ rows, but the old loop can spend m^2 candidate pairs on a coefficient known to vanish. The independent model verifies exact work differences. An eventual native benchmark should report support sizes, candidate-pair counts, frequency counts, achieved precision and whether the run completed, alongside wall time and memory.

A timing comparison that omits failed requests is misleading here: the main benefit can be that a request becomes possible under the same meaningful budget. Likewise, a benchmark that raises the budget only for the baseline changes the question. Report both the equal-budget availability comparison and the equal-result timing comparison as different experiments.

The candidate minimum-weight preflight also deserves its own counter. It can avoid a product when every resulting weight is outside the cutoff. That is not the same event as coefficient termination; conflating them would make it difficult to discover regressions in either condition.

12.2 Do not turn the backend fix into unnecessary symbolic work

Resolving one effective backend default is conceptually cheap compared with an asymptotic computation. Nevertheless, the implementation should not obtain the answer by repeatedly querying or evaluating every delayed option. A request record should retain the selector once it has been resolved. Instrumented counter tests are more informative than microbenchmarks for this particular correctness boundary.

A no-op explicit selector must not cause a second source evaluation. A metadata query about the selected route must not itself run the selected route again. These are semantic work counts that can be tested deterministically before attempting wall-time optimization.

12.3 A chart fast path should eliminate, not hide, solver work

For the reciprocal-square case, sign proof plus a closed-form rule is preferable to an unconstrained real solver followed by rejection. A broader signed-monomial fast path can prove its inverse identity directly. Store the selected chart evidence at construction rather than merely caching an unexplained rule.

A request-local timeout and a conservative fallback remain appropriate for difficult sign or domain proofs. Failure to establish a sign within budget should produce an unsupported/undecided chart result, not a guess. The scalar multiplication can then use the current composite envelope without pretending the conversion was proved. This preserves correctness while allowing the inexpensive, already provable cases to remain ordered.

12.4 Build and regression scope

The repository maintains modular source and a generated standalone. Production repairs should be applied to the canonical modules, followed by the repository's build procedure and selected acceptance tests against both loading paths. The supplied stager deliberately targets a disposable source copy so that reviewers can inspect a small candidate before touching the build workflow.

A load test after a rename is useful but not sufficient evidence for changes in held evaluation, option precedence or Fourier recurrence scheduling. Conversely, an older broad acceptance record should not be dismissed simply because it predates the rename: it remains historical evidence for its pinned source. The engineering task is to attach each result to the right revision and execution path, not to combine all counts into a larger-looking total.[5, 3]

13 Prioritized development packages

The following work packages are deliberately narrower than the previously recorded general roadmap.

First: effective backend ownership. Integrate the canonical-default correction behind the N01 counterexamples. Decide the alias policy before claiming the defect is closed for both public names. Add explicit/omitted/defaulted/invalid selector cases and preserve the native-versus-analytic result distinction. The acceptance gate is equivalent effective requests, not merely disappearance of an exception.

Second: selective option-key preparation. Review the N02 candidate together with the existing held-container tests. Require the full patched native package to pass key and delayed-value counter tests, plus specification-rule and embedded-rule negative controls. The candidate remains provisional until that integrated result exists; the current mechanism test is not sufficient for release.

Third: independent Fourier termination repair. Move the homogeneous zero check before the product while retaining an early support-exhaustion check. Add the budget-three private witness and public controls for nonzero frequencies and nonterminating coefficients. Record this explicitly as the Fourier extension of P03, rather than changing the earlier ordinary-engine status to imply coverage it did not have.

Fourth: a signed-coordinate closure increment. Begin with the reciprocal-square witness and its negative branch. Then add proved signed-monomial charts as a bounded family. Retain the branch, cutoff

mapping and error transform, and specify replay/refinement behavior. This work can improve actual algebraic usability without waiting for a general coefficient-ring or full transseries implementation.

These packages can be reviewed separately. The first and third have focused native candidate observations; the second still needs full integration testing; the fourth has a validated positive mechanism but additional lifecycle and branch obligations. None requires claiming a solution to all open items in the existing review register.

14 Source-coverage map and findings not promoted

14.1 Coverage map

The repository contains a modular kernel plus generated and validation artifacts. The following map states how the relevant areas entered this audit; it does not imply equal inspection depth in every file.

Area	Representative source	Audit use
Public entry and core jet machinery	<code>AsymptoticAnalysis.wl</code>	Public loading; core excerpts; context and representation checks.
Native entry and alias	<code>NativeCompatibility.wl</code>	Detailed source inspection; public baseline and selected candidate evaluations.
Shared series representation	<code>SeriesOperations.wl</code>	Coordinate inversion, regular representation and derivative/refinement excerpts.
Ordinary arithmetic	<code>SeriesArithmetic.wl</code>	Held normalization, regular operands and fallback routing.
Composite errors	<code>SeriesEnvelopeArithmetic.wl</code>	Error transport, approach handling and conservative fallback.
Fourier coefficients	<code>FourierCoefficients.wl</code>	Recurrence/convolution inspection; private native failure and candidate control.
Dirichlet tails	<code>DirichletSpecialFunctions.wl</code>	Full adapter formulas, coordinate construction and independent Lerch proof.
Exact special identities	<code>SpecialFunctionIdentities.wl</code>	Inspection of exact-before-asymptotic normalization and retained domains.
Special inverse adapters	<code>SpecialFunctionAdapters.wl</code>	Focused stored-option and replay inspection.
Gamma, Barnes, logarithmic and flat families	Corresponding modules listed in the kernel map	Architectural and contract comparison, using the current map, documentation and earlier-review register; no new broad native acceptance claim.

Area	Representative source	Audit use
Certificates and numeric checks	<code>InverseCertificates.wl</code> and related modules	Current status and evidence boundaries from the maintained register, rather than re-reporting repaired historical findings.
Tests, generated distribution and reports	<code>src/Tests</code> , <code>validation</code> , root standalone and <code>external-reports</code>	Revision provenance, selected native loading, nonduplication and reproduction design.

14.2 Examples of checks that did not become findings

An apparent option-precedence concern in special-inverse refinement was not promoted. The stored adapter-option list inspected in the source did not contain `"MaxTerms"`, so the particular hypothesis that it necessarily shadowed the later caller budget was not supported. Reporting it would have confused a plausible-looking pattern with an actual defect.

The Lerch tail formula itself was not found wrong. Its coefficient and moment construction agree with the independent derivation for the witness. Similarly, the composite product envelope is valid; N03 concerns lost ordered closure. The exact special-function identities inspected were not treated as wrong merely because some dominant asymptotic factors terminate.

The Fourier private witness does not establish that every public Fourier constructor has a low-budget regression. An existing public inverse/residual control continued to work. Finally, service failures in additional inverse-comparison and computed-key integration attempts are excluded from the package's defect count. These distinctions prevent the audit from increasing its finding count by relabeling unverified suspicions.

15 Artifact guide and reproduction

The archive is organized into the article, code, evidence and notices. `article/audit.tex` is self-contained and uses an inline bibliography; `article/audit.pdf` is the compiled version. Compilation needs an ordinary LaTeX installation with the listed packages, not a Wolfram kernel.

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error audit.tex
pdflatex -interaction=nonstopmode -halt-on-error audit.tex
```

The source patcher creates a fresh candidate and can optionally write an ordinary diff:

```
python code/stage_patches.py \
  --source /path/to/pinned/AsymptoticAnalysis.wl \
  --output /tmp/AsymptoticAnalysis-candidate.wl \
  --patches backend-default fourier-termination \
  --diff /tmp/candidate.diff
```

Adding `computed-keys` opts into the less-validated parser candidate. It should not be smuggled into a release merely because the stager accepts its unique textual anchor. The chart pilot is a separate temporary-kernel mechanism, not part of this textual patch set.

In a fresh Wolfram kernel, the observation runner can load the pinned source itself:

```
Get["code/run_observations.wl"]
```

To inspect a staged candidate instead, set `$AuditPackagePath` to its absolute local path before loading the runner. Set `$AuditOutputPath` to an output JSON path to avoid ambiguity about the working directory. Run baseline and candidate in separate fresh kernels so changed defaults, definitions and contexts do not leak between them. The observation runner records outputs; the desired-contract test file is separate.

The independent local checks need only Python:

```
python code/independent_checks.py \  
--output evidence/model_checks.json
```

A network-enabled environment can reproduce the lexical report scan with `code/corpus_screen.py`. That screen is intentionally labelled lexical. It does not establish semantic novelty, and a failed fetch is not interpreted as an empty document.

16 Conclusion

The pinned package has a defensible specialist role: it attaches useful algebraic, branch and error structure to real generalized expansions, while newer native delegation admits results that should not be assigned that analytic contract. The most consequential additional defect is at the interface between those roles: a backend default does not actually determine the effective request. Computed option keys reveal a second preparation gap.

The two algebraic contributions are narrower but substantive. Signed chart information already retained by the package can recover ordered arithmetic for the reciprocal-square Lerch example, with a direct proof and a quantitative bound for the original function. The Fourier engine still performs a useless product after homogeneous coefficient termination; the private native witness isolates that work and a focused candidate removes it without changing the requested exact answer.

These results support a development strategy based on precise boundaries: effective options, local charts, homogeneous recurrence termination and explicit evidence scope. The report does not claim a new unsound remainder or a complete repair of the existing review backlog. It supplies concrete additional failures and closure opportunities, mathematical justification for the proposed changes, and artifacts that make the next acceptance decisions reproducible.

References

- [1] Asymptotic Contributors. *AsymptoticAnalysis README*, reviewed revision 6687962f3c858a4f93623cfc496f33e35c6763d4. [Pinned repository README](#).
- [2] Asymptotic Contributors. *Kernel module map*. [Pinned module documentation](#).
- [3] Asymptotic Contributors. *Code review status and implementation register*. [Pinned status register](#). References to C, P, D, X and B identifiers in this article use this register's numbering.
- [4] Asymptotic Contributors. *External code reviews, wave two*. [Pinned review index](#). The corresponding [review archive](#) contains the earlier reports and their source articles.
- [5] Asymptotic Contributors. *Native expansion compatibility: required coverage and implementation plan*. [Pinned compatibility document](#).
- [6] Asymptotic Contributors. *NativeCompatibility.wl*. [Pinned source](#). Relevant routines: `expansionDispatch`, `nativeComputedArgumentQ`, `nativeSelectorValues`, `nativeExpansion`, and the held alias.
- [7] Asymptotic Contributors. *SeriesOperations.wl*. [Pinned source](#); especially [coordinate conversion and exact expression jets](#), `seriesData`, `seriesMake`, `seriesDerivative` and refinement replay.
- [8] Asymptotic Contributors. *SeriesArithmetic.wl*. [Pinned source](#). Relevant routines: `seriesRegularOperand`, `seriesArithmeticBinary` and held normalization.
- [9] Asymptotic Contributors. *SeriesEnvelopeArithmetic.wl*. [Pinned source](#). Relevant routines: `seriesEnvelopeBinary`, `seriesEnvelopePower` and `seriesEnvelopeUnary`.
- [10] Asymptotic Contributors. *FourierCoefficients.wl*. [Pinned source](#). Relevant routines: `fourierComposeBlock`, `fourierJetMul`, `fourierEuler` and `FourierInverseResidual`.
- [11] Asymptotic Contributors. *DirichletSpecialFunctions.wl*. [Pinned source](#). The independent proof in Section 7 checks a concrete case of its Lerch moment expansion.
- [12] Asymptotic Contributors. *SpecialFunctionIdentities.wl*. [Pinned source](#).
- [13] Wolfram Research. *Series*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Series.html>. Accessed September 9, 2026.
- [14] Wolfram Research. *SeriesData*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/SeriesData.html>. Accessed September 9, 2026.
- [15] Wolfram Research. *Asymptotic*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Asymptotic.html>. Accessed September 9, 2026.
- [16] Wolfram Research. *InverseSeries*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>. Accessed September 9, 2026.
- [17] Wolfram Research. *AsymptoticSolve*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>. Accessed September 9, 2026.
- [18] Wolfram Research. *SetOptions*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/SetOptions.html>. Accessed September 9, 2026.
- [19] Wolfram Research. *OptionsPattern*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/OptionsPattern.html>. Accessed September 9, 2026.
- [20] NIST Digital Library of Mathematical Functions. *Section 25.14: Lerch's Transcendent*. <https://dlmf.nist.gov/25.14>. Defining sum used in (7); accessed September 9, 2026.